

Universidade de Vigo

Deseño e desenvolvemento dunha funcionalidade de
busca e filtrado de cursos no catálogo de cursos en liña

Aarón Riveiro Vilar

Traballo Fin de Grao
Escola de Enxeñaría de Telecomunicación
Grao en Enxeñaría de Tecnoloxías de Telecomunicación

Titores:
Manuel Caeiro Rodríguez
Óscar Vázquez Trigo

Curso 2025/2026

Índice

1. Introducción	1
1.1. Contexto do proxecto	1
1.2. Motivación	1
1.3. A plataforma de partida	2
2. Obxectivos e requisitos	2
2.1. Requisitos funcionais	3
2.1.1. Ordenación de resultados	3
2.1.2. Busca difusa	3
2.2. Requisitos non funcionais	4
3. Deseño	4
3.1. Ordenación de resultados	4
3.2. Busca difusa	5
4. Desenvolvemento	6
4.1. Ordenación de resultados	6
4.2. Busca difusa	7
5. Conclusións e liñas futuras	8
6. Referencias	8
7. Anexos	10
A. Estado da arte	10
A.1. Busca difusa e correspondencia aproximada de cadeas	10
A.2. Busca e indexación en sistemas de xestión de contidos	10
B. Calibrado do algoritmo de busca difusa	11

1. Introducción

Nos últimos anos, o crecemento sostido dos sistemas de información dixitais deu lugar a repositorios con volumes de datos cada vez maiores e máis heteroxéneos. Neste contexto, a mera dispoñibilidade da información xa non resulta suficiente: é necesario dotar os sistemas de mecanismos eficaces que permitan localizar, filtrar e presentar os datos de forma relevante para o usuario. A disciplina da recuperación de información (*Information Retrieval*) estuda precisamente os métodos e modelos que permiten acceder de maneira eficiente a grandes coleccións de documentos, maximizando a relevancia dos resultados ofrecidos fronte a unha consulta determinada [1].

A medida que aumenta o tamaño e a complexidade dos repositorios, a experiencia de usuario pasa a depender en grande medida da calidade do sistema de busca e filtrado. Non só é importante a rapidez de resposta, senón tamén a precisión e exhaustividade dos resultados, así como a capacidade do sistema para interpretar a intención de busca do usuario e ofrecer mecanismos de refinamento progresivo mediante filtros, facetas ou criterios adicionais [2]. Un sistema de busca limitado a coincidencias literais sobre un único campo de texto pode resultar insuficiente en contornas nas que os datos presentan múltiples dimensións e atributos relevantes.

Por iso, o deseño e a implementación de estratexias avanzadas de busca e filtrado constitúen un elemento clave en calquera sistema de información orientado ao usuario final, especialmente cando se pretende facilitar o acceso eficiente a contidos estruturados e mellorar a usabilidade global da plataforma.

1.1. Contexto do proxecto

O proxecto DACEM (*Digitising Academic Catalogues for Enhanced Mobility*) [3], aprobado na convocatoria de 2023 do programa Erasmus+ para Asociacións Estratéxicas en Educación Superior (código de proxecto 2023-1-ES01-KA220-HED-000160344), ten como finalidade o desenvolvemento dunha plataforma de catálogo de cursos dirixida a institucións educativas europeas. O seu obxectivo principal é ofrecer un sistema que permita publicar e manter información actualizada sobre titulacións e materias de forma accesible, estruturada e interoperable, facilitando así os procesos de mobilidade académica internacional. A plataforma está orientada a dous perfís principais de usuarios: as institucións educativas, que a empregan para publicar e manter actualizado o seu catálogo de cursos; e os estudantes que, no marco de procesos de mobilidade Erasmus, consultan a oferta formativa das universidades parceiras.

O sistema de información asociado a esta plataforma está fundamentado en tecnoloxías de software libre e está concibido para poder despregarse tanto en contornas locais como en infraestruturas na nube. Máis alá do almacenamento estruturado de datos académicos, o correcto funcionamento do sistema depende da súa capacidade para xestionar, organizar e recuperar a información de maneira eficiente. Neste contexto, os mecanismos de busca e filtrado desempeñan un papel esencial, xa que constitúen o principal medio de interacción entre o usuario e o repositorio de cursos. A calidade destes mecanismos condiciona directamente a usabilidade do sistema, a precisión dos resultados obtidos e a experiencia global de navegación.

1.2. Motivación

Un dos retos fundamentais nos sistemas de información que xestionan grandes volumes de datos estruturados non reside unicamente no seu almacenamento, senón na forma en que estes datos son recuperados e presentados ao usuario. A medida que un repositorio medra en número de elementos e en complexidade de atributos, os mecanismos de busca simples baseados en coincidencias textuais directas poden resultar insuficientes para ofrecer resultados relevantes e axustados ás necesidades reais de consulta.

Esta problemática é especialmente significativa en contornas nas que os datos presentan múltiples dimensións descritivas (como titulacións, materias, áreas de coñecemento, idio-

mas, créditos ou modalidades) e nas que o usuario pode ter criterios de busca diversos e combinados. A ausencia de mecanismos avanzados de filtrado e refinamento progresivo limita a capacidade do sistema para adaptarse a diferentes perfís de usuario e escenarios de consulta, reducindo a eficiencia na localización de información pertinente.

No ámbito actual do desenvolvemento de aplicacións web e sistemas de xestión de contidos, as tendencias apuntan cara á implementación de motores de busca máis sofisticados, capaces de indexar múltiples campos, ponderar resultados segundo a relevancia, incorporar criterios de filtrado dinámico e mellorar a experiencia de usuario mediante interfaces intuitivas. Estas solucións non só optimizan o acceso á información, senón que contribúen a unha maior usabilidade e aproveitamento do sistema no seu conxunto.

O presente Traballo de Fin de Grao enmárcase neste contexto, propoñendo o deseño e desenvolvemento dunha mellora nos mecanismos de busca e filtrado do catálogo de cursos da plataforma DACEM.

1.3. A plataforma de partida

A plataforma DACEM está implementada sobre Drupal 10, un sistema de xestión de contidos (CMS) de código aberto amplamente empregado no ámbito académico e institucional [4]. Drupal 10 proporciona unha contorna flexible para a definición de tipos de contido estruturado, a xestión de usuarios e a publicación de información a través de interfaces web configurables.

O modelo de datos organízase arredor de tres tipos de contido principais: *programme*, que representa unha titulación académica; *individual educational component* (IEC), correspondente a unha materia ou compoñente curricular individual; e *institution*, que modela as universidades participantes. Cada entidade conta cun conxunto de campos estruturados que recollen atributos como a área de coñecemento, o nivel de cualificación, os créditos, o idioma de impartición ou o país de orixe, entre outros.

Para a presentación e consulta do catálogo, a plataforma emprega o módulo *Views*, integrado no núcleo de Drupal, que permite definir listaxes e consultas de contido de forma declarativa. O módulo *Better Exposed Filters* [5] complementa *Views* proporcionando a interface de filtrado ao usuario. Existen dúas vistas principais de busca: *search_programme*, accesible en `/search-programme`, orientada á consulta de titulacións; e *search_iec*, en `/search-iec`, orientada á consulta de compoñentes educativos individuais. A capa de presentación visual está baseada nun tema personalizado construído sobre Bootstrap 5.

O mecanismo de busca textual actual baséase no filtro *combine* de *Views*, que realiza comparacións de subcadeas sobre o campo título das entidades. Esta aproximación presenta limitacións relevantes:

- A busca restrínxese ao campo título, ignorando atributos como a descrición ou a área de coñecemento.
- Non se aplica ningunha ponderación por relevancia, polo que os resultados se presentan en orde alfabética con independencia da pertinencia da consulta.
- Non existe tolerancia a erros tipográficos, de xeito que calquera inexactitude na consulta pode producir cero resultados aínda que o elemento buscado exista no catálogo.
- As opcións de filtrado son estáticas e non se adaptan ao conxunto de resultados dispoñibles, polo que poden aparecer valores sen correspondencia con ningunha entrada existente.

2. Obxectivos e requisitos

O presente traballo ten como obxectivo principal mellorar a experiencia de busca e navegación do catálogo de cursos da plataforma DACEM. Partindo das limitacións identificadas no sistema actual (ausencia de criterios de ordenación, busca por coincidencia exacta e sen

tolerancia a erros tipográficos), propónse o deseño e implementación de dúas melloras concretas: a incorporación de criterios de ordenación de resultados nas vistas de busca existentes, e a implementación dun mecanismo de busca difusa que permita localizar programas e compoñentes educativos aínda en presenza de erros tipográficos na consulta.

A continuación defínense os requisitos funcionais e non funcionais que guían o desenvolvemento. Enténdese por requisito funcional aquel que describe as capacidades concretas que debe ofrecer o sistema, mentres que os requisitos non funcionais establecen condicións relativas ao seu comportamento, integración e mantenibilidade, sen especificar funcionalidades concretas.

2.1. Requisitos funcionais

2.1.1. Ordenación de resultados

1. O sistema debe permitir ao usuario ordenar os resultados das vistas de busca segundo diferentes criterios, adaptados ao tipo de entidade que se consulta.
2. Cada criterio de ordenación debe permitir alternar entre orde ascendente e descendente mediante a mesma acción de usuario.
3. Ambas as vistas deben contemplar os seguintes criterios de ordenación comúns: orde alfabética por título, área de coñecemento, número de créditos e país de orixe da institución.
4. A vista de programas (*search_programme*) debe incluír adicionalmente o criterio de ordenación por nivel de cualificación EQF.
5. A vista de compoñentes educativos (*search_iec*) debe incluír adicionalmente os criterios de ordenación por programa asociado e cuadrimestre.
6. A interface de ordenación debe integrarse visualmente de forma coherente co deseño existente da plataforma.

2.1.2. Busca difusa

1. O sistema debe ser capaz de localizar resultados relevantes aínda cando a consulta introducida polo usuario conteña erros tipográficos ou grafías aproximadas.
2. A busca difusa debe aplicarse a ambas as vistas de busca: a vista de programas e a vista de compoñentes educativos.
3. A busca difusa debe aplicar un criterio de similaridade entre a consulta e os títulos dos elementos do catálogo, de maneira que se poidan recuperar resultados con pequenas desviacións ortográficas respecto ao texto almacenado.
4. A normalización da consulta debe incluír a conversión a minúsculas e a eliminación de diacríticos, de xeito que maiúsculas e caracteres acentuados non afecten aos resultados da busca.
5. A busca debe operar sobre todos os idiomas indexados, de xeito que o usuario poida localizar resultados con independencia do idioma no que estea configurada a interface ou do idioma no que estea almacenado o título.
6. Os resultados deben presentarse ordenados por relevancia, de xeito que os elementos con maior correspondencia coa consulta aparezan en primeiro lugar.
7. O mecanismo de busca difusa debe integrarse no fluxo de consulta existente, de xeito que o usuario non perciba diferenzas na interface respecto ao comportamento anterior.
8. O sistema debe manter un comportamento consistente ante consultas baleiras ou moi curtas, evitando erros ou resultados inesperados.

2.2. Requisitos non funcionais

1. A solución debe integrarse coa plataforma existente sen alterar a estrutura fundamental do modelo de datos nin comprometer o funcionamento do resto de funcionalidades do sistema.
2. O desenvolvemento debe estar fundamentado en tecnoloxías de código aberto, mantendo coherencia cos principios e a arquitectura do proxecto DACEM.
3. O sistema debe garantir un comportamento estable e previsible ante entradas incorrectas ou consultas mal formadas, evitando erros visibles para o usuario final.
4. A interface debe cumprir criterios básicos de usabilidade, asegurando que os mecanismos de interacción resulten intuitivos e coherentes co deseño xeral da plataforma.
5. O sistema debe garantir tempos de resposta axeitados nas consultas, de maneira que as melloras introducidas non degraden de forma perceptible a experiencia de uso.
6. A solución debe minimizar o impacto nos recursos do servidor, evitando sobrecargas innecesarias e optimizando as consultas realizadas á base de datos.
7. O rendemento do sistema debe manterse estable ante un incremento razoable do volume de datos almacenados no catálogo, asegurando que a solución escale de forma adecuada.
8. A implementación debe estruturarse de forma modular e documentada, de xeito que cada mellora poida ser mantida ou ampliada de forma independente sen afectar ao resto do sistema.

3. Deseño

3.1. Ordenación de resultados

Para incorporar a funcionalidade de ordenación á vista de programas e á vista de compoñentes educativos, optouse por empregar o mecanismo de ordenación exposta do módulo *Views* de Drupal, en lugar de implementar unha solución baseada en manipulación directa de consultas ou en ordenación no lado do cliente. Esta decisión responde a tres razóns principais:

- O mecanismo intégrase de forma nativa co sistema *AJAX* de *Views*, o que evita ter que reimplementar a actualización parcial de resultados.
- Mantén a coherencia coa arquitectura da plataforma, que xa emprega *Views* de forma extensiva para a xestión e presentación de contido.
- Os criterios de ordenación quedan declarados na configuración da vista en lugar de estar codificados en PHP, o que facilita a súa modificación sen necesidade de modificar código.

O mecanismo funciona do seguinte xeito: cando se declara un criterio de ordenación como exposto na configuración dunha vista, *Views* xera automaticamente dous campos de control no formulario asociado, un que identifica o criterio polo que ordenar e outro que indica a dirección (ascendente ou descendente), e reconstrúe a consulta ao recibir o envío do formulario.

Para a interacción do usuario, deseñouse un comportamento de alternancia: o primeiro clic sobre un botón de ordenación establece o criterio en orde ascendente; un segundo clic sobre o mesmo botón inverte a dirección de ordenación, como se mostra na figura 1. Esta decisión permite concentrar cada criterio nun único botón, evitando duplicar os controis en dúas versións por dirección. A lóxica de alternancia, implementada en *JavaScript*, le o campo de ordenación activo no formulario e inverte a dirección se coincide co botón pulsado, ou establece a orde ascendente por defecto en caso contrario.

Os criterios de ordenación definidos para cada vista son os seguintes:



Figura 1: Botóns de ordenación dispoñibles na vista de programas.

- **Vista de programas:** título, área de coñecemento, nivel EQF, créditos e país.
- **Vista de compoñentes educativos:** título, programa pai, créditos, cuadrimestre, área de coñecemento e país.

Os criterios de tipo textual (título, área de coñecemento, programa pai e país) ordénanse de forma alfabética; os de tipo numérico (créditos, nivel EQF e cuadrimestre) ordénanse por valor numérico.

Algúns destes criterios implican campos que non pertencen directamente ao nodo listado, senón a entidades relacionadas. O país reside no nodo institución referenciado polo programa; a área de coñecemento e o título do programa pai na vista de compoñentes educativos pertencen ao nodo programa referenciado polo IEC. Nestes casos, a ordenación resólvese a través das relacións entre entidades xa declaradas na configuración da vista, sen necesidade de definir unións adicionais entre táboas.

3.2. Busca difusa

A busca difusa articulouse en tres compoñentes, aplicados de forma idéntica á vista de programas e á vista de compoñentes educativos:

- **Índices de Search API:** un índice para cada tipo de contido (programas e compoñentes educativos), que proporcionan o catálogo de títulos sobre o que opera a clase de puntuación.
- **Hooks de Views:** dous puntos de extensión de Drupal que integran a lóxica de busca difusa na execución das vistas.
- **Clase PHP de puntuación:** calcula a similitude entre a consulta e cada título indexado mediante a distancia de Levenshtein.

Fronte a unha consulta SQL directa ás táboas de nodos, os índices ofrecen a vantaxe de desacoplar a lóxica de puntuación do esquema da base de datos e de manterse actualizados de forma automática ao modificarse o contido. Ambos os índices están configurados para

cubrir todos os idiomas dispoñibles, de xeito que a busca non queda restrinxida ao idioma da interface.

A decisión de interceptar a consulta mediante *hooks*, en lugar de actuar sobre a capa de presentación, responde ao obxectivo de que o mecanismo sexa transparente para o usuario e compatible co sistema *AJAX* de *Views*.

A puntuación aplícase a nivel de palabra, e non sobre a frase completa, porque nun título formado por varias palabras un único erro tipográfico impediría calquera coincidencia ao comparar as cadeas enteiras. Comparando palabra a palabra, cada termo avalíase de forma independente e o resultado global reflicte mellor o grao real de similitude. Tanto o termo de busca como cada título do índice son normalizados previamente mediante conversión a minúsculas e eliminación de acentos, o que garante que a busca sexa insensible a maiúsculas e acentos, conforme ao requisito funcional establecido na sección 2. A continuación, tokenízanse en palabras de tres ou máis caracteres; o limiar de lonxitude mínima descarta preposicións, artigos e partículas curtas que, pola súa alta frecuencia, xerarían falsos positivos. Para cada par (palabra da consulta, palabra do título) calcúlase unha puntuación de similaridade segundo os seguintes niveis:

- Coincidencia exacta: puntuación 1,0.
- Coincidencia de prefixo (a palabra da consulta é inicio dunha palabra do título): puntuación α (pendente).
- Distancia de Levenshtein 1: puntuación proporcional á lonxitude das palabras (pendente).
- Distancia de Levenshtein 2 (só para palabras suficientemente longas): puntuación menor (pendente).
- Distancia superior ao limiar: puntuación 0,0 (descartado).

A puntuación total dun elemento é a suma das mellores coincidencias obtidas para cada palabra da consulta. Esta escala continua permite ordenar os resultados por relevancia, a diferenza dun enfoque binario que simplemente incluíría ou excluíría resultados sen distinguir grao de axuste. Os valores concretos de cada nivel, incluído α e os limiares de distancia, foron determinados de forma empírica a través dun proceso de calibrado descrito no anexo B (pendente).

O fluxo de execución é o seguinte: cando o usuario introduce un termo, o *hook* de pre-execución intercepta a consulta antes de que *Views* a execute. Nese momento invoca a clase de puntuación, que carga todos os títulos indexados en *Search API*, normaliza e tokeniza tanto o termo de busca como cada título, e calcula a puntuación de cada candidato mediante a distancia de Levenshtein. Con esa información, elimina a condición de filtrado por subcadea exacta e substitúea por un filtro polo conxunto de identificadores dos elementos que superaron o limiar de puntuación. A continuación, *Views* executa a consulta modificada e o *hook* de post-execución reordena os resultados por puntuación descendente, dado que filtrar por un conxunto de identificadores non garante ningunha orde determinada. As puntuacións compártense entre ambos os *hooks* mediante un mecanismo de almacenamento compartido, que actúa como canle de comunicación entre as dúas fases de execución.

4. Desenvolvemento

4.1. Ordenación de resultados

A implementación da ordenación articulouse en tres capas coordinadas:

- A configuración declarativa das vistas.
- As plantillas Twig.
- A lóxica JavaScript.

Na configuración YAML de cada vista, cada criterio de ordenación declarouse como exposto e asignóuselle un identificador único (*field identifier*). Este identificador establece o contrato entre a configuración da vista e a capa JavaScript: o valor do atributo `data-sort-by` de cada botón no HTML debe coincidir exactamente con este identificador para que o mecanismo funcione correctamente.

Algúns criterios implican campos de entidades relacionadas; nestes casos, o criterio declara o parámetro de relación correspondente, reutilizando as unións entre entidades xa definidas na configuración da vista, sen necesidade de declarar unións adicionais:

- **Vista de programas:** a ordenación por país emprega a relación coa institución referenciada polo programa.
- **Vista de compoñentes educativos:** a ordenación por área de coñecemento e por título do programa pai usa a relación co nodo programa referenciado polo IEC; a ordenación por país encadéase ademais a través da relación co nodo institución.

Nas plantillas Twig de cada vista engadíronse os botóns de ordenación, cada un coa clase `js-sort` e o atributo `data-sort-by` co identificador do criterio correspondente. As plantillas de ambas as vistas cargan a mesma biblioteca JavaScript.

A plataforma de partida xa incluía un script JavaScript con lóxica de detección de clics sobre os botóns `.js-sort` e o envío do formulario. Modificouse para implementar o comportamento de alternancia descrito na sección 3.1: en lugar de usar sempre a dirección especificada no atributo do botón, o script le agora o estado actual dos campos `sort_by` e `sort_order` do formulario e inverte a dirección se o mesmo criterio xa está activo, ou establece a dirección ascendente por defecto en caso contrario. Así mesmo, actualizouse o método de envío: en lugar de disparar un evento *change* sobre os campos do formulario, o script simula agora un clic directo no botón de envío, o que desencadea o mecanismo de *Views* e actualiza os resultados mediante AJAX sen recargar a páxina.

4.2. Busca difusa

A implementación da busca difusa concretouse en tres elementos:

- A configuración do servidor e dos índices de *Search API*.
- A extensión do módulo `custom_views_filters` mediante dous *hooks* de *Views*.
- A nova clase `FuzzySearch` que encapsula a lóxica de puntuación.

Creouse un servidor de *Search API* con *backend* de base de datos, configurado cun limiar de lonxitude mínima de palabra de 3 caracteres, coherente co mecanismo de tokenización da clase de puntuación; a indexación de frases e a concordancia parcial desactiváronse, xa que o índice se emprega unicamente como catálogo de títulos sobre o que operar a métrica de Levenshtein, sen usar a busca nativa do módulo. Sobre este servidor configuráronse dous índices: `programmes`, que indexa os nodos de tipo *programme* polo campo `title`, e `courses`, que indexa os nodos de tipo *course* polo mesmo campo. Ambos os índices se configuraron sen restrición de idioma.

O *hook* `hook_views_query_alter` invócase antes de que *Views* execute a consulta, permitindo interceptala e modificala. Este *hook* xa existía no módulo `custom_views_filters` para outras vistas; engadiuse nel a lóxica de busca difusa para a vista de programas e a vista de compoñentes educativos. Para cada vista, obtense o valor do campo `combine` do formulario exposto e invócase a clase `FuzzySearch` co índice correspondente. A condición SQL orixinal do filtro `combine` identifícase polo operador `formula` e a presenza do termo de busca nos seus argumentos, e elimínase da *query*. En función do resultado: se a clase devolve `NULL` (o termo non produce tokens), non se aplica ningún filtro e móstranse todos os resultados; se devolve un array baleiro, engádese a condición `nid IN (0)` para forzar un resultado baleiro; se hai coincidencias, as puntuacións gárdanse para o *hook* de post-execución e engádese a condición `nid IN ({NIDs})`.

Engadiuse ademais o novo *hook* `hook_views_post_execute`, que se invoca xusto despois de que *Views* execute a consulta. Este comproba se o array de puntuacións non está baleiro

e, de ser así, reordena os resultados da vista por puntuación descendente e limpa o array para evitar interferencias en execucións posteriores.

A clase `FuzzySearch`, creada dentro do módulo `custom_views_filters`, centraliza a lóxica de puntuación. O punto de entrada é o método estático `scoreFromIndex()`, que recibe o texto de busca e o identificador do índice e devolve os candidatos ordenados por puntuación descendente, ou `NULL` se o texto non produce ningún token válido. Este delega en dous métodos: `loadCandidatesFromIndex()`, que obtén os candidatos do índice, e `scoreAndFilter()`, que aplica a puntuación. A clase mantén ademais a propiedade estática `$scores`, que actúa como canle de comunicación entre os dous hooks: o hook de pre-execución escribe as puntuacións e o de post-execución léera.

`loadCandidatesFromIndex()` consulta o índice de *Search API* mediante unha *query* sen restrición de idioma e cun rango de ata 10.000 resultados. O identificador de cada ítem ten o formato `entity:node/{nid}:{lang}`; o NID extráese deste identificador mediante expresión regular.

`scoreAndFilter()` normaliza e tokeniza o termo de busca e, para cada candidato, compara cada palabra da consulta con cada palabra do título, acumulando a mellor puntuación obtida. Dado que *Search API* indexa cada tradución dun nodo como un ítem independente, o mesmo NID pode aparecer varias veces; por iso, os resultados indexanse por NID e, en caso de duplicado, consérvase a puntuación máis alta.

A tokenización divide o texto en palabras de 3 ou máis caracteres mediante a expresión regular `[a-z]{3,}`. O método `wordScore()` compara dúas palabras normalizadas e devolve unha puntuación entre 0 e 1. Os primeiros casos son a coincidencia exacta, que retorna unha puntuación de 1, e a coincidencia de prefixo, que retorna unha puntuación de α . Para os demais, calcúlase a distancia de Levenshtein mediante a función `levenshtein()` de PHP e compárase cun limiar (isto é, a cantidade máxima de erros permitidos): se o supera, a puntuación é 0; se non, aplícase a fórmula $1 - d/L$, onde d é a distancia e L a lonxitude da maior palabra das dúas, de xeito que máis erros implican menor puntuación. O limiar é 1 para palabras de ata 5 caracteres e 2 para as máis longas.

5. Conclusións e liñas futuras

(pendente)

6. Referencias

- [1] J. Murel and M. Syed, “What is information retrieval?” <https://www.ibm.com/think/topics/information-retrieval>, consultado: abril de 2026.
- [2] “Retrieval accuracy,” <https://www.sciencedirect.com/topics/computer-science/retrieval-accuracy>, consultado: abril de 2026.
- [3] “Digitising academic catalogues for enhanced mobility,” <https://projects.uni-foundation.eu/dacem/>, European University Foundation, 07/2024, consultado: abril de 2026.
- [4] W3Techs, “Usage statistics and market share of Drupal,” <https://w3techs.com/technologies/details/cm-drupal>, consultado: xuño de 2026.
- [5] M. Keran *et al.*, “Better Exposed Filters,” https://www.drupal.org/project/better_exposed_filters, consultado: xuño de 2026.
- [6] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [7] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.
- [8] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in *Proceedings of the Section on Survey Research Methods*. American Statistical Association, 1990, pp. 354–359.
- [9] E. Ukkonen, “Approximate string-matching with q-grams and maximal matches,” *Theoretical Computer Science*, vol. 92, no. 1, pp. 191–211, 1992.

- [10] T. Seidl *et al.*, “Search API,” https://www.drupal.org/project/search_api, 2024, consultado: maio de 2026.
- [11] Apache Software Foundation, “Apache Solr,” <https://solr.apache.org>, consultado: maio de 2026.
- [12] Elastic, “Elasticsearch,” <https://www.elastic.co/elasticsearch>, consultado: maio de 2026.

7. Anexos

A. Estado da arte

A.1. Busca difusa e correspondencia aproximada de cadeas

A busca difusa (*fuzzy search*) é unha técnica de recuperación de información que permite localizar resultados relevantes aínda cando a consulta do usuario non coincide exactamente co texto almacenado. A diferenza da busca por subcadeas exactas, tolera desviacións ortográficas e erros tipográficos, o que mellora a experiencia de usuario en sistemas con datos introducidos manualmente ou con vocabulario técnico susceptible a equivocacións.

O fundamento teórico da busca difusa reside no concepto de distancia de edición entre dúas cadeas, definida como o número mínimo de operacións elementais necesarias para transformar unha cadea na outra. O algoritmo máis empregado é a **distancia de Levenshtein** [6], que considera tres operacións básicas: inserción, eliminación e substitución dun carácter. Dados dous textos de lonxitude m e n , calcúlase en tempo $O(m \times n)$ mediante programación dinámica.

A **distancia de Damerau-Levenshtein** [7] amplía este modelo incorporando as transposicións de caracteres adxacentes como operación atómica, o que a fai máis precisa para modelar erros tipográficos humanos, onde o intercambio de dúas letras consecutivas é un dos erros máis frecuentes. Outra métrica habitual é a **similitude de Jaro-Winkler** [8], que premia as cadeas con prefixo común e adoita empregarse na comparación de nomes propios. Por último, os enfoques baseados en **n-gramas** [9] dividen as cadeas en secuencias solapadas de n caracteres e comparan a proporción de n-gramas compartidos, ofrecendo bo rendemento en textos longos e sendo facilmente paralelizables.

Para o presente traballo optouse pola distancia de Levenshtein como criterio de similaridade, pola súa sinxeleza conceptual, a súa robustez ante os tipos de erro tipográfico máis comúns e a súa idoneidade para cadeas curtas como os títulos de programas e compoñentes educativos.

A.2. Busca e indexación en sistemas de xestión de contidos

Os sistemas de xestión de contidos modernos incorporan habitualmente mecanismos de busca que van máis alá da simple consulta SQL. En Drupal 10, a solución integrada por defecto para a busca en listaxes de contido é o módulo *Views*, cuxo filtro *combine* realiza comparacións de subcadeas mediante operadores SQL LIKE. Esta aproximación límitase a coincidencias exactas e non ofrece capacidades de indexación nin de ponderación de resultados por relevancia.

Para cubrir estas limitacións, o ecosistema de Drupal conta co módulo *Search API* [10], que proporciona unha capa de abstracción para a indexación e consulta de contido. O módulo inclúe un sistema de procesadores configurables (como normalización de maiúsculas, transliteración de acentos, tokenización ou eliminación de palabras baleiras) que se aplican durante a indexación con independencia do *backend* escollido. Os *backends* dispoñibles máis empregados son os seguintes:

- **Backend de base de datos:** almacena o índice en táboas da base de datos relacional existente. Non require infraestrutura adicional e ofrece un rendemento axeitado para catálogos de tamaño moderado.
- **Apache Solr** [11]: motor de busca de código aberto baseado en Lucene, optimizado para grandes volumes de datos e con capacidades avanzadas de ponderación de resultados por relevancia. Require un servidor Solr externo.
- **Elasticsearch** [12]: solución distribuída orientada á alta dispoñibilidade e á escalabilidade horizontal, con capacidades similares a Solr. Tamén require infraestrutura adicional dedicada.

Para este traballo optouse polo *backend* de base de datos, dado que non require infraestrutura adicional, é suficiente para o volume de datos do catálogo DACEM, e permite integrar

a lóxica de busca difusa directamente en PHP, empregando o índice para recuperar os títulos sobre os que aplicar a métrica de Levenshtein.

B. Calibrado do algoritmo de busca difusa

(pendente)